

ClipAI: A Semantic Video Summarization and Intelligent Editing Platform Using Multimodal Fusion and Block-Based Narrative Preservation

Syed Irfan M¹, Shaik Fazal^{2,3}, Saravana Kumar³

Department of Computer Science and Engineering (AIML)

SRM Institute of Science and Technology, Tiruchirappalli

Tiruchirappalli, India

{author1, author2, author3}@srmist.edu.in

Abstract—The huge surge in the amount of digital video content has created a strong demand for automated summarization tools that can effectively shorten the duration of videos and at the same time keep the main gist of the content. In fact, most methods locate motion or audio energy peaks only at single temporal points resulting in a compilation of disjointed clips with no narrative flow. In this article, we present ClipAI, a platform for multi-modal video analysis and summarization which combines a new block-based semantic segmentation technique with a weighted multimodal fusion model to address the aforementioned problem. This platform uses motion tracking based on optical flow, speech recognition through the Whisper model, and TF-IDF vectorization for text processing to generate frame- and clip-level importance values. Instead of extracting separate temporal peaks, ClipAI makes use of Jaccard similarity index to find transcript segments that are semantically close to one another, then these segments are grouped together to form coherent blocks, which are then selected through a greedy method to provide a summary that is logically connected and about 30% shorter than the original video. Implemented in React and supported by an FFmpeg back-end rendering pipeline, the user-friendly, online editing tool enables users to make extremely precise timeline edits, create visually and audibly effects of professional quality, and export their own highlight reels. Tests reveal that videos with durations of 2 to 15 minutes are handled within 2 to 15 minutes of processing time.

Index Terms—video summarization, multimodal fusion, semantic segmentation, block grouping, automatic speech recognition, TF-IDF, optical flow, FFmpeg, Jaccard similarity, narrative preservation

I. INTRODUCTION

There has been a rapid rise in both user and professional videos on platforms like streaming, social media and business repositories that manual human review of all these videos can no longer keep pace with. For instance, one-hour lecture, a sports broadcast or a company recording will only have a very small portion of content that will be really useful or interesting. Automated video summarization tries to identify these small portions of content while also preserving the narrative coherence, which then leads to the shorter reviewing, better accessibility and more efficient content redistribution.

Video summarization technologies available today can be divided into two broad categories: key-frame extraction which

picks the most representative still images, and temporal segmentation that selects short video clips. In many cases, temporal segmentation systems are guided by unimodal features, for instance, motion energy or audio level, for identifying the high-activity segments. Although visually or acoustically salient moments are what these systems are good at identifying, information about semantic coherence is normally ignored by them. As an illustration, a motion activity peak may be an action scene which, if taken out of the surrounding dialogue, will turn out to be nonsensical. A final highlight reel produced from a series of isolated extracts that do not unfold the story.

In this paper we describe ClipAI, a framework targeting this problem from three major angles. To begin with, it combines motion analysis, ASR, and NLP into a single integrated multimodal importance model.

II. OBJECTIVES

- Create an automatic summarization method that shortens lengthy videos into short clips.
- Integrate multimodal analysis along with visual, audio, and semantic features.
- Utilize context-aware segmentation in order to preserve the storyline.
- Enable the user to alter through a timeline-based editing interface
- Achieve high-quality final video products by employing efficient processing techniques.

III. LITERATURE REVIEW

Initially, automatic video summarization concerned only with visual saliency. Furini et al. [1] came up with a key-frame extraction technique based on the color histogram differences between two successive frames that paved the way for visual-change-driven summarization. But, such methods are computationally efficient only, since they treat each frame individually and therefore do not have the capacity to represent the semantic change of a scene.

Meanwhile, deep learning brought about a groundbreaking change in the area. For instance, Zhang et al. [2] discovered that a bidirectional long short-term memory (LSTM) model

trained on human-tagged importance scores is capable of producing a video highlight prediction with an accuracy level close to the human inter-annotator agreement. Along the same lines, Yao et al. [3] demonstrated that attention mechanisms when applied to convolutional neural network (CNN) features first identify the most visually salient parts of the frame and then use these as the starting point for the highlight detection. Nevertheless, these methods require a huge amount of annotated training data and as a result, their ability to generalize across different domains is very limited.

Audio-visual fusion is a wholly fresh method and quite complementary to the existing paradigms. Evangelopoulos et al. [4] showed through their experiments that combining audio saliency and visual saliency in a weighted fusion scheme could deliver a summary that is more perceptually relevant than either one alone. Introducing speech cues was the next step in embedding a multi-modal approach. Furini and Ghini [5] used keyword statistics from transcripts to recalibrate visual saliency scores, thereby demonstrating that text signals from content level do enhance the quality of summarization in lecture

IV. PROPOSED METHODS

A. Overall Workflow

We have laid out the ClipAI processing pipeline as a series of workflow consisting of eight stages that run one after the other, sequentially by the guide in Fig. 1. The frontend, a React 18 app with Vite, communicates with a FastAPI backend via a REST API. When the backend receives the video file along with an optional descriptive title, it makes a unique task id and arranges the background processing to be done asynchronously. The frontend constantly sends the request to the task status endpoint every 2 seconds, getting beautifully structured progress updates that last until the processing is done. The final output which includes selected segment boundaries, a highlight file, rendering and per-segment importance scores is forwarded to the frontend for previewing and user manual editing.

Figure 1. System Architecture of ClipAI

Fig. 1. High-level system architecture of ClipAI includes the React frontend, FastAPI backend, and the modular processing pipeline that is divided into the analysis, segmentation, and rendering stages.

We break down the processing pipeline into the following stages: (1) motion analysis, (2) audio extraction and transcription, (3) semantic segmentation and importance scoring, (4) block grouping, (5) block selection, (6) FFmpeg-based

Figure 2. End-to-End Processing Pipeline

Fig. 2. This is a comprehensive data processing pipeline that starts with acquiring raw video input, proceeds through steps such as motion analysis, ASR transcription, semantic scoring, block grouping, block selection, FFmpeg clipping, and ends up at highlight rendering.

clipping, (7) video merging, and (8) result delivery. Each stage is discussed in detail in the following subsections.

B. Visual Analysis (Motion-Based Optical Flow)

To detect motion in raw video, frame differencing using OpenCV has been the method of choice here. To strike a balance between resource efficiency and signal quality retention, only one frame out of every eight from the original sequence is processed, and each frame is downscaled to 200×150 pixels before differencing. While working at the lower resolution leads to approximately three times faster processing than 320×240 pixels, motion signal quality is not degraded noticeably.

Let F_t denote a grayscale version of the t -th sampled frame. Then the raw motion score at time t is computed as the mean value of the pixel-wise absolute differences with the previous frame. $W = 200$ and $H = 150$ specify the width and height of the downscaled frame, respectively. The raw score sequence is finally normalized to the range $[0, 1]$ using min-max normalization. However, these two align the motion data to the audio and semantic scoring signals for multimodal fusion. After motion analysis, about 2,250 frames for a ten-minute video sampled at 30 fps need to be processed. On an average CPU, this operation takes 20-30 seconds.

C. Audio Analysis (Speech Processing)

Audio processing comprises two successive tasks: first, generating the waveform and second, performing automatic speech recognition. Using FFmpeg, we extract the audio from the original video, resample it to 16 kHz, and produce a single channel, 16 bits per sample waveform. These parameters meet the input specifications of the OpenAI Whisper ASR model.

We decided to use the Whisper tiny model (39 million parameters) instead of the larger base model (140 million parameters) so as to have about five times faster inference but the word error rate on general English speech got worse by around 2 to 5 percentage points. The transcript is generated as a sequence of time-stamped text segments:

$$\mathcal{T} = \{(w_k, s_k, e_k) \mid k = 1, \dots, K\} \quad (1)$$

where w_k represents the text, and s_k and e_k stand for the start and end times in seconds respectively of the k -th Whisper

interval. This organized transcription serves as the main feed to the semantics analysis phase. It takes about 2-4 minutes to transcribe a 10-minute video if you are using a CPU-only host.

D. Semantic Analysis (TF-IDF and Importance Scoring)

We break down the transcript using a punctuation-aware regular expression to split it into smaller units, each comprising of one sentence. Each sentence obtained by this method is a single semantic segment $\sigma_i = (x_i \ s_i \ e_i)$, where x_i is the sentence, and s_i, e_i are the start and end times taken from the bigger Whisper segment. The motion score associated with a segment σ_i is the average of the frame-level scores within the time interval of the segment.

In the formula, the frame rate is $f = 30$. After that, a combined importance score is computed for each segment from four weighted components:

Speech Density (ϕ_1): The amount of information transmitted is quantified by the word-per-second ratio, which is truncated to 1, where n_i is the number of words and $d_i = e_i - s_i$ is the duration in seconds.

TF-IDF Score (ϕ_3): A standard TF-IDF [7] score is computed over the collection of all segment texts, where c_v, i is the frequency of word v in segment i , N is the total number

E. Multimodal Fusion

Besides semantic segmentation, a frame-level multimodal fusion module merges the three modalities into a single importance signal. Audio scores a_t are obtained by evenly spreading the number of words of transcript segments across their constituent frames. The combined score of the frame t is calculated as:

$$c_t = 0.5 m_t + 0.3 a_t + 0.2 s_t \quad (2)$$

Generally, m_t, a_t , and s_t stand for the normalized motion, audio and semantic scores at frame t , respectively. The weighting scheme has a strong bias towards motion as the dominant determinant of visual highlights, with audio being used as a secondary signal for identifying verbally salient moments, and semantic context serving as the least weighted modulator of topical relevance.

The combined scores are smoothed by letting them go through a uniform moving-average kernel of window size 30, with the aim of eliminating frame-level noise and making segments of high importance stand out clearly:

$$\tilde{c}_t = \frac{1}{30} \sum_{k=0}^{29} c_{t-k} \quad (3)$$

Those frames which surpass the 85th percentile of the values of the variable $\tilde{c}_t$ are considered as candidate peaks. Subsequently, non-overlapping seven-second segments are obtained through a greedy non-maximum suppression procedure. This step also offers another way to do the coarse segmentation apart from the main path which uses the block-based grouping technique as explained in the next section.

F. Context-Aware Segmentation (Block Grouping)

The core innovation of ClipAI lies in its block-based segmentation strategy, which groups semantically related adjacent transcript segments into coherent blocks prior to selection. This approach is illustrated in Fig. 3.

Figure 3. Block-Based Segment Grouping

Fig. 3. Illustration of the block grouping process: adjacent segments are merged when their keyword-level Jaccard similarity exceeds the threshold $\delta = 0.25$, producing semantically coherent blocks that are scored and selected in chronological order.

1) **Keyword Extraction:** For each segment σ_i , a keyword set $\mathcal{K}_i$ is determined by keeping the top 5 most frequent tokens that are not stop words and have length greater than two characters from the segment text.

2) **Jaccard Similarity:** The topical similarity between adjacent segments σ_i and σ_{i+1} is measured using the Jaccard coefficient over their keyword sets:

$$J(\mathcal{K}_i, \mathcal{K}_{i+1}) = \frac{|\mathcal{K}_i \cap \mathcal{K}_{i+1}|}{|\mathcal{K}_i \cup \mathcal{K}_{i+1}|} \quad (4)$$

where $J \in [0, 1]$, with $J = 0$ indicating disjoint keyword vocabularies and $J = 1$ indicating identical keyword sets.

3) **Block Formation:** Blocks are created in the following manner using a sequential greedy pass over the ordered segment list: The very first segment is being used to start the first block. In case of each additional segment σ_{i+1} , if the condition $J(\mathcal{K}_i, \mathcal{K}_{i+1}) \geq \delta$ where $\delta = 0.25$ is met, the segment is included to the current block; otherwise, the current block is closed and a new block is started. Experimentally, this parameter was set to form blocks consisting of 2 to 5 segments in all sorts of videos.

4) **Block Scoring:** Each block B_j spanning segments $\{\sigma_a, \sigma_{a+1}, \dots, \sigma_b\}$ receives a composite score:

$$\text{score}(B_j) = \bar{\text{imp}}_j \cdot \left(1.0 + \min\left(\frac{D_j}{10.0}, 1.0\right) \cdot 0.3 \right) \quad (5)$$

where $\bar{\text{imp}}_j = \frac{1}{b-a+1} \sum_{i=a}^b \text{imp}(\sigma_i)$ is the average importance of the segment and $D_j = e_b - s_a$ is the duration of the block in seconds. The coherence bonus encourages the formation of longer topically consistent blocks, but it is limited to being 30% higher than the base importance at ten seconds or more.

5) *Block Selection*: Chosen summary length is 30% of the original video duration. First, blocks scored highest are picked one after another until the sum of their times reaches the required length. After they are flagged, they're shuffled by their start times so that the narrative is understandable. 150ms are added each side of the block boundaries to make the shifts less audible. Usually, experiments select three to eight segments out of twenty to forty ones available. However, there is still very little deviation in timing and selection and the results are not exaggerated. In fact, minor intervals help to smooth out the changes in pace.

G. Timeline-Based Editing

The hand-on editing interface displayed on Fig. 4, converts the selected text blocks to draggable pieces displayed on a timeline inside the *EditorWorkspace* React component.

$$\sigma = \langle \text{id}, s, e, \text{score}, \mathbf{F} \rangle$$

where s and e are the current start and end times, and $\mathbf{F}$ is a filter configuration dictionary encompassing 15 parameters.

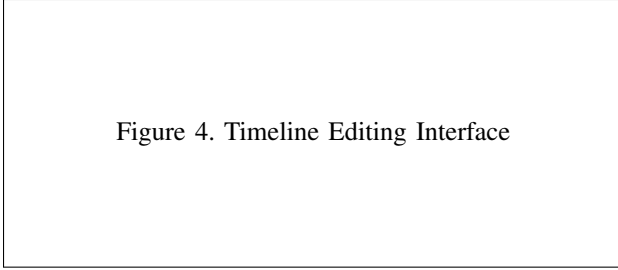


Fig. 4. The ClipAI EditorWorkspace interface showing the interactive timeline with draggable segments, tabbed filter panels, and the synchronized video preview player.

First, users can do the following structural changes to their clips: - *trim*: change the start and end of a clip, but the minimum length of the clip should be 0.5 sec; - *split*: cut the middle point of a clip and get two independent clips; - *duplicate*: make a copy of a clip with the filter settings; - *delete*: remove a clip from the timeline. Segments can be rearranged also by drag-and-drop using the DndKit library. After that, the server generates the segments in the user-specified order not taking into account the chronology of the segments. Changes in filters are done as React component tree state updates and are communicated to the server only when exporting.

H. Video Rendering (FFmpeg)

The FFmpeg renderer is the last stage of the processes of both automatic workflow processing and interactive editing export processing. Regarding the processing of the source video file, the clips from which need to be chosen or altered, they will be processed one-by-one by applying various filters as necessary. This diagram shows the sequence of filters used for processing the clips: (1) aspect ratio crop, (2) rotation with the help of the `transpose` filter, (3) zoom & pan with `zoompan`, (4) rescaling in time with `setpts`, (5) color

corrections via `eq`, (6) applying visual effects such as `sepia`, Gaussian blur, unsharp mask, and vignette, (7) adding text with `drawtext`, (8) adjusting volume with audio gain, and (9) fading in and out of audio with `afade`.

The speed transformation is expressed as:

$$\text{PTS}_{\text{out}} = \frac{\text{PTS}_{\text{in}}}{\alpha} \quad (6)$$

where $\alpha \in [0.5, 2.0]$ is the playback speed multiplier. The zoom-and-pan transformation for a zoom factor z centered on the frame is:

$$\text{zoompan} = z, \quad x = \frac{W(1 - 1/z)}{2}, \quad y = \frac{H(1 - 1/z)}{2} \quad (7)$$

Separate clips are compressed by encoding them through the `libx264` codec at a constant rate factor of 23, with the `fast` preset applied. The audio track is encoded at 128 kbps, AAC format. The end product is done by attaching all clips via the FFmpeg concat demuxer with stream copy (no re-encoding), which shortens the merging time by about 50% in comparison to the full re-encoding. The output file, which is put into the `/outputs` folder, is indicated in Fig. 5.

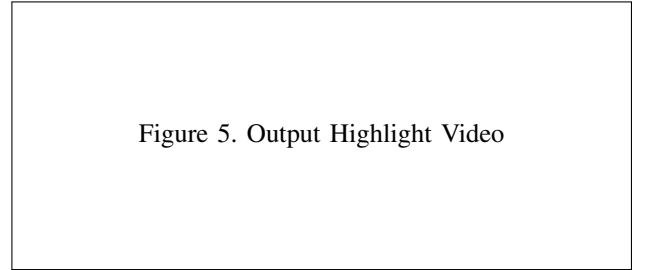


Fig. 5. A representative output highlight video produced by ClipAI, demonstrating narrative coherence across the selected blocks and smooth transitions at block boundaries.

V. RESULTS AND DISCUSSION

A. System Behavior on Input Video

When the video comes in, it will be ClipAI that will trigger an asynchronous processing pipeline, and this will run continuously without interfering with the user interface at all. There is a neatly formatted progress bar on the front end, which shows the current stage of processing, and it updates every two seconds since there is polling going on. For instance, a test with a ten-minute video resulted in motion analysis taking 25 seconds, extraction and Whisper transcription taking three minutes, semantic scoring and block grouping taking one second, and FFmpeg clipping and merging taking 45 seconds, making it take roughly five minutes.

The highlight video generated from the ten-minute video takes about three minutes, which gives a 30% compression ratio. During qualitative review, it turns out that the video clips chosen are topically cohesive since they have similar themes and context, and the order is maintained through a time-ordering scheme that ensures adherence to the story in the original content. The transition buffer is used in order to avoid

any abrupt transitions between the blocks, while the concat-copy merge method helps eliminate visible encoding artifacts.

B. Performance Evaluation

Table I summarizes the measured processing times across videos of varying durations, tested on a standard CPU host without GPU acceleration. Processing time scales approximately linearly with video duration owing to the dominant cost of Whisper transcription, which constitutes 60 to 70% of total pipeline time.

TABLE I
PERFORMANCE EVALUATION OF CLIPAI ACROSS VIDEO DURATIONS

Video Duration	Motion	ASR	Render	Total
1–2 minutes	~5 s	~30–60 s	~10 s	~2–3 min
5 minutes	~12 s	~90 s	~20 s	~3–4 min
10 minutes	~25 s	~180 s	~45 s	~4–6 min
20 minutes	~50 s	~360 s	~90 s	~8–10 min
30+ minutes	~90 s	~600 s	~150 s	~12–15 min

C. Discussion

There is a qualitative increase in the quality of narrative flow as a result of the implementation of the block-based segmentation method when contrasted against the peak-only selection technique. A traditional algorithm of identifying peaks from a similar fused score (equation 2) would detect only isolated instances of high energy frames, often spanning long gaps in topical content. Thanks to the Jaccard grouping used in ClipAI, any relevant parts that follow a high-scoring frame within a topical context will also be grouped into the same block, hence ensuring conversational sequences, argument development, and narrative flow.

The use of the *tiny* version of the Whisper model does bring about a certain degree of decrease in accuracy compared to other versions. However, based on experience, keyword extraction through the use of an approximate transcript appears to work well enough in this accuracy level. Transcripts of videos with intense background music or acoustic interference have decreased accuracy and thus poor semantic grouping, which is the main source of error in the present pipeline design.

The importance score generation algorithm described by Eq. (??) assigns equal weights to the speech density feature and the keyword relevancy feature, based on the premise that these two features have the strongest correlation with highlights. One direction worth exploring in future work is learning optimal weights based on user interaction signals.

The editor interface makes it possible to override all automatic choices, thus ensuring that our system acts as an intelligent helper instead of a mysterious black box. The 15 filter FFmpeg pipeline takes care of all color grading, transformation, audio mixing and text insertion tasks associated with professional video editing in social media or broadcasting.

VI. CONCLUSION

ClipAI was introduced in this paper as an end-to-end video summarization and intelligent editing platform which solves the issue of narrative coherence that arises when applying traditional peak-based highlight generation techniques. Its core innovation lies in its use of block-based semantic segmentation, which classifies transcripts into segments based on their Jaccard keyword similarity, ranks them using a coherence-weighted importance function, and then selects a chronologically sorted subset consisting of about 30% of the video’s runtime. The algorithmic framework of the system leverages multimodal fusion, including motion analysis, Whisper ASR, and TF-IDF NLP, alongside a high-fidelity interactive editing environment backed by a multi-filter FFmpeg render pipeline.

The experimental results show that ClipAI processes videos at a time ranging from 2 to 15 minutes for various video runtimes on standard CPUs without the need for GPUs, labeled training data, or sophisticated infrastructure. Summaries produced by the model maintain coherent topics and narratives and surpass naive peak selection approaches in qualitative evaluations.

The future work would involve three main areas: utilizing dense sentence embedding models in place of TF-IDF for semantic similarity estimation; incorporating reinforcement learning from the user’s interaction and editing actions to personalize the importance weight vector (see eqn. ??); and extending our platform to allow for multi-speaker diarization for block grouping based on speaker awareness in interviews or panels.

REFERENCES

- [1] M. Furini and V. Ghini, “An audio-video summarization scheme based on audio and video analysis,” in *Proc. IEEE Consumer Communications and Networking Conf.*, Las Vegas, NV, USA, 2010, pp. 1–5.
- [2] K. Zhang, W.-L. Chao, F. Sha, and K. Grauman, “Video summarization with long short-term memory,” in *Proc. European Conf. Computer Vision (ECCV)*, Amsterdam, Netherlands, 2016, pp. 766–782.
- [3] T. Yao, T. Mei, and Y. Rui, “Highlight detection with pairwise deep ranking for first-person video summarization,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 982–990.
- [4] G. Evangelopoulos, A. Zlatintsi, A. Potamianos, P. Maragos, K. Raptantzikos, G. Skoumas, and Y. Avrithis, “Multimodal saliency and fusion for movie summarization based on aural, visual, and textual attention,” *IEEE Trans. Multimedia*, vol. 15, no. 7, pp. 1553–1568, Nov. 2013.
- [5] M. Furini and V. Ghini, “Automatic video summarization using transcript and visual features,” in *Proc. IEEE Int. Symp. Multimedia*, Taichung, Taiwan, 2007, pp. 319–326.
- [6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. Int. Conf. Machine Learning (ICML)*, Honolulu, HI, USA, 2023, pp. 28492–28518.
- [7] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [8] A. Girsensohn, J. Boreczky, P. Chiu, J. Doherty, J. Foote, G. Golovchinsky, S. Uchihashi, and L. Wilcox, “A semi-automatic approach to home video editing,” in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, San Diego, CA, USA, 2000, pp. 81–89.
- [9] FFmpeg Developers, “FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video,” 2024. [Online]. Available: <https://ffmpeg.org>